

COMP2271: Computer Graphics

Part 5: Building a World

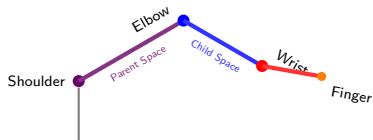
Dr. Frederick Li

Department of Computer Science
Durham University

The Proprioception Paradox

How your brain calculates geometry

Before we look at code, I want you to close your eyes and touch the tip of your nose.



The Math:

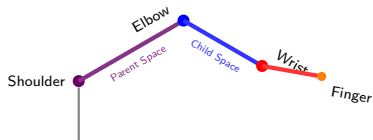
$$P_{finger} = M_{shoulder} \times M_{elbow} \times M_{wrist}$$

The Proprioception Paradox

How your brain calculates geometry

Before we look at code, I want you to close your eyes and touch the tip of your nose.

How did you do that? You didn't see your hand. You didn't calculate the global (x, y, z) coordinate of your fingertip relative to the floor.



The Math:

$$P_{finger} = M_{shoulder} \times M_{elbow} \times M_{wrist}$$

The Proprioception Paradox

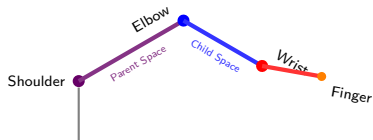
How your brain calculates geometry

Before we look at code, I want you to close your eyes and touch the tip of your nose.

How did you do that? You didn't see your hand. You didn't calculate the global (x, y, z) coordinate of your fingertip relative to the floor.

Your brain solved a **Kinematic Chain**:

- 1 It rotated your **Shoulder** (Parent).
- 2 Relative to that, it rotated your **Elbow** (Child).
- 3 Relative to that, it rotated your **Wrist** (Grandchild).



The Math:

$$P_{finger} = M_{shoulder} \times M_{elbow} \times M_{wrist}$$

To your brain, your fingertip is not a point in the world. It is a child of the arm hierarchy.

The Proprioception Paradox

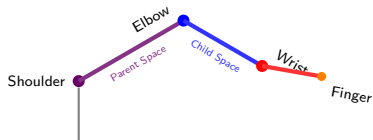
How your brain calculates geometry

Before we look at code, I want you to close your eyes and touch the tip of your nose.

How did you do that? You didn't see your hand. You didn't calculate the global (x,y,z) coordinate of your fingertip relative to the floor.

Your brain solved a **Kinematic Chain**:

- ① It rotated your **Shoulder** (Parent).
- ② Relative to that, it rotated your **Elbow** (Child).
- ③ Relative to that, it rotated your **Wrist** (Grandchild).



The Math:

$$P_{finger} = M_{shoulder} \times M_{elbow} \times M_{wrist}$$

To your brain, your fingertip is not a point in the world. It is a child of the arm hierarchy.

The Insight

We are about to learn how to teach the computer to think like your brain: **Hierarchically**.

- 1 **The Problem of Complexity:** Why managing many objects is hard.
- 2 **The Scene Graph:** Our solution for organising a 3D world.
- 3 **Hierarchical Transformations:** The mathematics of parent-child relationships.
- 4 **Practical Scenarios:** Applying scene graphs to real-world examples.
- 5 **Implementation:** How it works in code (CPU vs. GPU).
- 6 **Three.js in Practice:** Bridging theory with a high-level library.

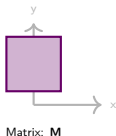
Where we are

So far, we have focused on rendering **individual objects**. We can define their geometry, apply transformations (MVP matrix), and simulate light to make them look 3D.

The Next Step: Building a World

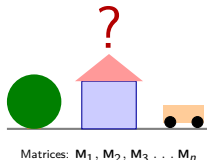
A real 3D scene (a game, a simulation) contains **hundreds or thousands** of objects. How do we manage them all?

1. The Single Object



Scaling Up
----->

2. The Complex World



The Problem with a "Flat List"

Why manual synchronization fails at scale

The "Bag of Objects" Approach

Storing every entity in a linear array or list:

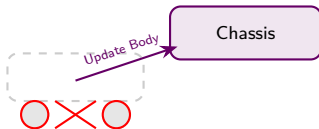
- `scene = [chassis, w1, w2, door];`
- Each object has its own **Global** (x, y, z).

The "Maintenance" Nightmare

Relationships are non-existent:

- **Manual Sync:** Moving the car requires updating the translation of every part individually.
- **Relative Motion:** Spinning a wheel while driving requires complex world-space trigonometry.

Visualising the "Broken" Link



Wheels are "Stuck" in
Global Space

"In a flat list, the computer doesn't know the wheel belongs to the car."

A Better Way: A Hierarchy

Instead of a flat list, we organise objects in a **hierarchy** or tree-like structure. This is the core idea of a Scene Graph.

Key Idea

A **Scene Graph** is the fundamental data structure used to organise the objects in a 3D scene by representing their relationships. It transforms chaos into a structured world.

Hierarchies are natural:

- A solar system has planets, which have moons.
- A person has arms, which have hands, which have fingers.
- A car has a chassis, which has wheels.

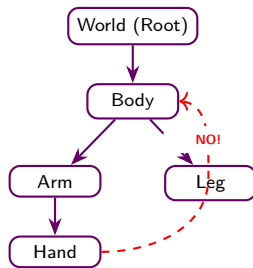
What is a Scene Graph? The Core Definition

Scene Graph Definition

A **Scene Graph** is a hierarchical data structure, specifically a *directed acyclic graph* (DAG), used to organize and manage the objects within a 3D scene. Every element in the scene (a model, a light, a camera) is represented as a **node** in this tree.

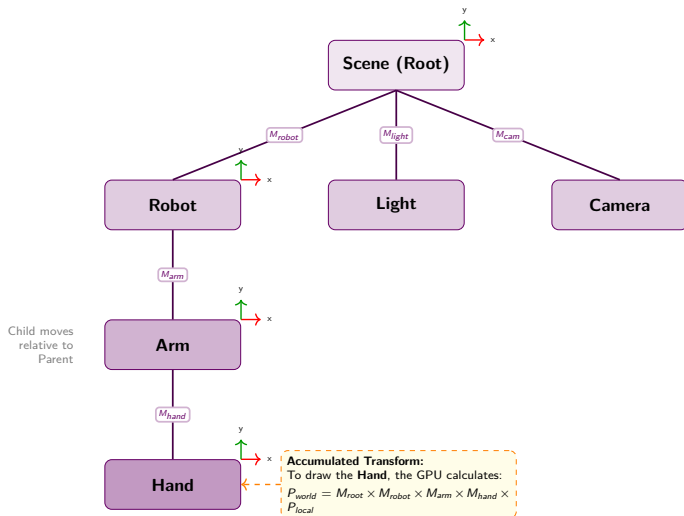
Key Terminology

- **Directed:** Relationships are one-way, from **parent** to **child**.
- **Acyclic:** A node cannot be its own parent or ancestor. This prevents infinite loops!
- *Example: A hand is a child of the arm, but the arm cannot be a child of the hand.*



Hierarchical Structure

The Scene Graph Hierarchy of a 3D Scene

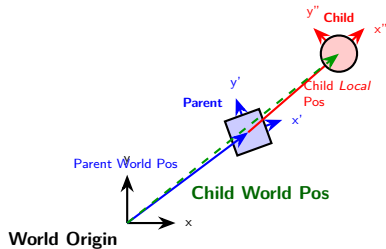


Each edge represents a **Local Transformation Matrix**. The final position of the "Hand" is the result of multiplying all matrices along the path from the Root.

The power of the scene graph comes from how it handles transformations.

Local vs. World Space

- Each node stores its transformation (pos, rot, scale) **relative to its parent**. This is its **Local Transformation**.
- The final position in the scene is its **World Transformation**.



$$\vec{P}_{\text{world}} = \mathbf{M}_{\text{parent}} \times \vec{P}_{\text{local}}$$

The Core Principle

An object's final world position is determined by its own local transformation **AND** the transformations of all its ancestors.

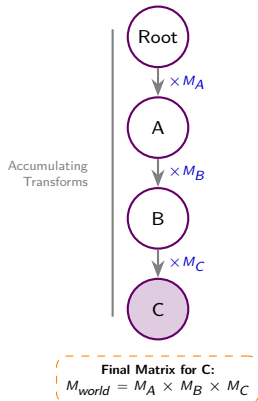
The Magic: Concatenating Transformations

We calculate a node's final world transformation by multiplying matrices up the hierarchy.

The Core Formula

For any node, the final world matrix is:

$$M_{world} = M_{parent_world} \times M_{local}$$



We calculate a node's final world transformation by multiplying matrices up the hierarchy.

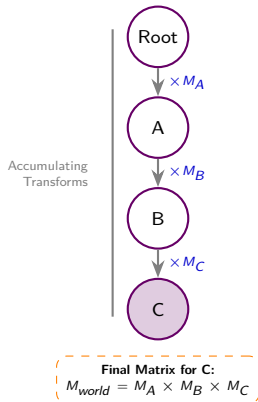
The Core Formula

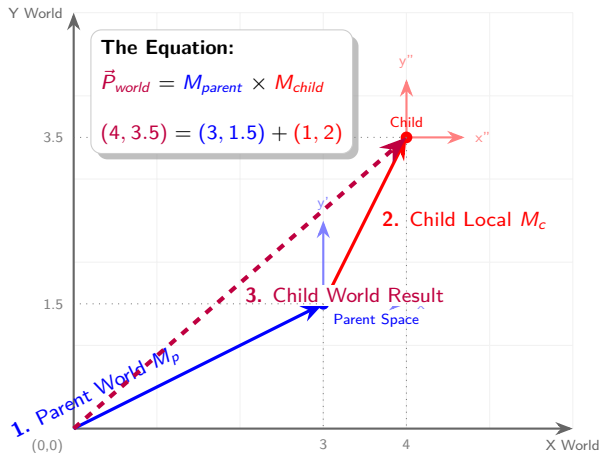
For any node, the final world matrix is:

$$M_{world} = M_{parent_world} \times M_{local}$$

Extensions & Optimizations

- **Recursion:** This expands all the way to the root: $M_{final} = M_{root} \times M_{body} \times M_{arm} \times M_{hand}$
- **"Dirty Flags":** Real engines optimise this! If a parent doesn't move, we don't re-multiply the matrices for the children.





The Child's Local Transform (Red) is defined **relative** to the Parent's axes, not the World's.

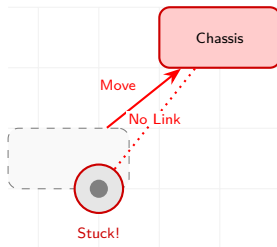
Why We Need It: The Comparison

Managing Complexity: Flat vs. Hierarchical

Without Scene Graph (Flat)

The "Bag of Objects" Approach

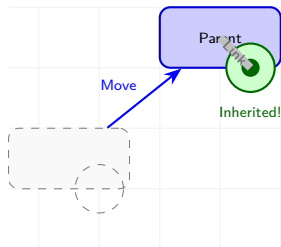
- Objects are independent.
- **The Problem:** Moving the Chassis does *not* automatically move the Wheels.
- **Result:** The vehicle breaks apart.



With Scene Graph (Hierarchy)

The "Tree" Approach

- Objects are linked (Parent → Child).
- **The Solution:** Moving the Chassis automatically carries the Wheels.
- **Result:** The vehicle stays intact.



Key Implication: Hierarchy ensures *logical consistency* during movement. ▶

Composition via Scene Graph

The Hierarchical Approach

With a Scene Graph, we define **Parent-Child relationships** instead of manually calculating the full chain. The engine recursively propagates transformations.

The Recursive Formula: Each object inherits its parent's transform.

$$M_{global_child} = M_{global_parent} \cdot M_{local_child}$$

- **How it works:**

- 1 **Parent (Car):** Move car in world.

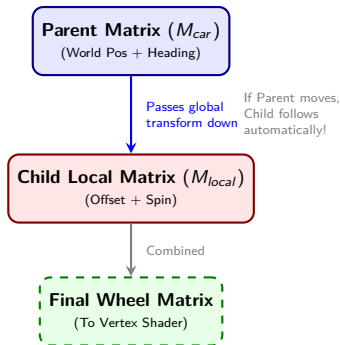
$$M_{car} = T_{world} \cdot R_{heading}$$

- 2 **Child (Wheel):** Define relative offset.

$$M_{local} = T_{offset} \cdot R_{spin}$$

- 3 **Auto-Composition:** Engine multiplies:

$$M_{final} = M_{car} \cdot M_{local}$$

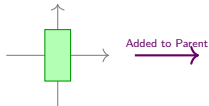


How Scene Graphs Simplify Logic

Separation of Concerns: You manage the parts, the Graph manages the whole.

With a Scene Graph, we treat complex motion as simple, isolated behaviors. We stop thinking about the "final world position" and focus on **Local State**.

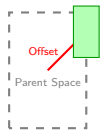
1. Child Behavior



Task: **Spin**

`wheel.rotation.x += 0.1`

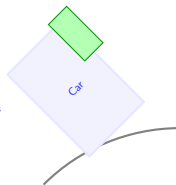
2. Static Setup



Task: **Mount**

`wheel.position.set(x,y,z)`

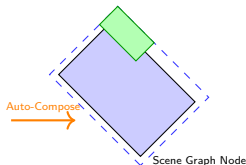
3. Parent Behavior



Task: **Drive**

`car.position.x += speed`

4. The Result



Complex Motion
Solved

The "Set and Forget" Advantage

You simply tell the wheel to **spin** and the car to **drive**. The Scene Graph automatically ensures the spinning wheel stays attached to the driving car.

SG App 1: Complex Animation (Forward Kinematics)

Propagating Motion through Hierarchies

Concept: Objects rarely move in isolation. Parts are attached to other parts (e.g., a hand attached to an arm).

Why use it? To animate a robot, you shouldn't need to calculate the hand's world position manually every frame. You just rotate the shoulder.

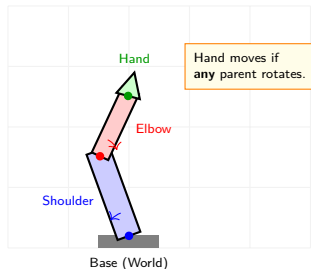
How it Operates:

- 1 **Local Transform:** Every node stores only its local offset/rotation relative to its parent.
- 2 **Traversal:** The engine walks down the tree multiplying matrices:

$$M_{hand} = M_{shoulder} \cdot M_{elbow} \cdot M_{wrist}.$$

Attention Needed: "Dirty Flags"

Recalculating every matrix every frame is slow. Engines use a "Dirty Flag": if a parent doesn't move, we don't re-compute the children.



SG App 2: Optimisation (Frustum Culling)

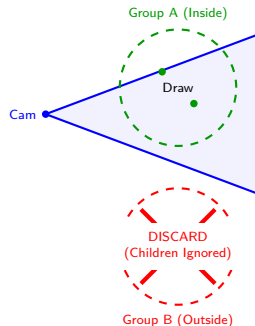
Pruning the Tree to Save GPU Cycles

Concept: "Don't process what you can't see."

Why use it? A city scene might have 1,000,000 objects. If the camera is inside a single room, checking every distant building individually kills performance.

How it Operates:

- 1 **Bounding Volumes:** Each Parent Node calculates a box/sphere that encapsulates *all* its children.
- 2 **The Check:** If the Parent's box is outside the Camera View (Frustum), we **discard the whole branch** immediately.



Attention Needed: Update Cost

If a child object moves, the Parent's Bounding Box must assume the new size. This propagates up the tree.

The "Selfie Stick" Revelation

The Scene Graph's Secret Weapon

We usually think of moving objects *in front* of the camera. But what if we treat the **Camera** as just another object?

The Scenario: You are building a racing game. You want the camera to follow the car, drift when it turns, and shake when it hits a bump.

The "Selfie Stick" Revelation

The Scene Graph's Secret Weapon

We usually think of moving objects *in front* of the camera. But what if we treat the **Camera** as just another object?

The Scenario: You are building a racing game. You want the camera to follow the car, drift when it turns, and shake when it hits a bump.

The Hard Way (Math): Calculate the car's velocity, rotation vector, add an offset, interpolate... every single frame.

The "Selfie Stick" Revelation

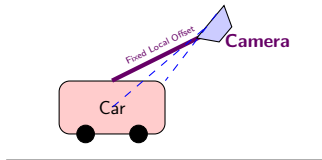
The Scene Graph's Secret Weapon

We usually think of moving objects *in front* of the camera. But what if we treat the **Camera** as just another object?

The Scenario: You are building a racing game. You want the camera to follow the car, drift when it turns, and shake when it hits a bump.

The Hard Way (Math): Calculate the car's velocity, rotation vector, add an offset, interpolate... every single frame.

The Easy Way (Hierarchy): Simply make the Camera a **child** of the Car. `car.add(camera);`



The "Selfie Stick" Revelation

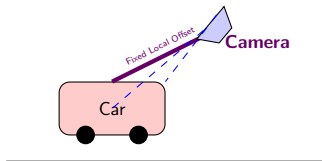
The Scene Graph's Secret Weapon

We usually think of moving objects *in front* of the camera. But what if we treat the **Camera** as just another object?

The Scenario: You are building a racing game. You want the camera to follow the car, drift when it turns, and shake when it hits a bump.

The Hard Way (Math): Calculate the car's velocity, rotation vector, add an offset, interpolate... every single frame.

The Easy Way (Hierarchy): Simply make the Camera a **child** of the Car. `car.add(camera);`



The Paradox

By bolting the camera to the object (like a digital selfie stick), the complex math of "following" disappears. If the car moves, the camera moves for free!

Practical Scenario 1: The Solar System

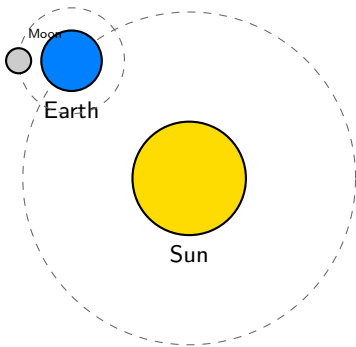
The solar system is the classic example of hierarchical modeling. Without a scene graph, calculating orbits requires complex trigonometry. **With a scene graph, global orbits emerge naturally from simple local rotations.**

The Hierarchy

- **Sun** (Child of Scene Root)
 - **Earth** (Child of Sun)
 - **Moon** (Child of Earth)

The Result

If you apply a **local rotation** to the Earth node, the Earth spins on its axis. Because the Moon is a child, it automatically **orbits** the spinning Earth!



Hierarchies make orbital mechanics intuitive.

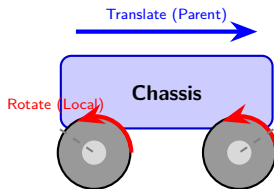
While the solar system shows orbital motion, a vehicle demonstrates **articulated motion**. We need distinct parts to move together as a cohesive group, while maintaining the freedom to animate independent parts (like spinning wheels) locally.

The Hierarchy

- **Car Chassis** (Parent)
 - **Wheel 1** (Child)
 - **Wheel 2** (Child)

Separation of Concerns

- 1 **Chassis Translation:** Moves the whole car. Wheels inherit this automatically.
- 2 **Wheel Rotation:** Spin locally. This does **not** affect the chassis.



$$P_{wheel} = M_{chassis} \times M_{spin} \times P_{local}$$

The scene graph is not just theoretical. It is the backbone of virtually all modern 3D graphics applications.

Game Engines

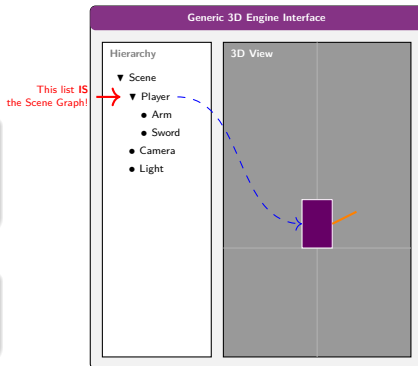
Engines like **Unity**, **Unreal**, and **Godot** are built around it. The "Hierarchy" panel is a direct visualisation of the graph.

3D Modeling Software

Blender, **Maya**, and **3ds Max** use it to manage relationships between meshes, bones, and lights.

Web Graphics

Libraries like **Three.js** use explicit graph manipulation methods like `parent.add(child)`.



When you drag-and-drop an item under another in a game engine, you are editing the graph structure.

A Critical Distinction

The Scene Graph is a **CPU-side abstraction**. Its purpose is to help us, the programmers, manage complexity.

The CPU's Job

- Store the graph structure.
- Traverse upwards.
- Multiply matrices to find the final **Model Matrix** (M_{world}).
- Send this *one* matrix to GPU.

The GPU's Job

- Receive a single, final Matrix.
- Knows *nothing* about hierarchy.
- Performs parallel vertex transformations ($M_{world} \times P$).



Hierarchical Logic

Flat Math

Code: Calculating the World Matrix

This pseudo-code shows the core calculation for a single node.

JavaScript Pseudo-code

```
// Assume the parent's world matrix is known
const parentWorldMatrix = getParentWorldTransform();

// The child's local matrix (relative to parent)
const childLocalMatrix = mat4.create();
mat4.translate(childLocalMatrix, childLocalMatrix, [5, 0, 0]);

// The final result is the multiplication of the two
const childWorldMatrix = mat4.create();
mat4.multiply(childWorldMatrix, parentWorldMatrix, childLocalMatrix);

// This 'childWorldMatrix' is now ready to be sent to the
// shader's u_modelMatrix uniform.
```

Analysis

The child's final position is not set independently; it is **derived** from its parent. If the parent moves, this calculation is re-run, and the child's world matrix is automatically updated.

Code: The Recursive Render Loop

To draw the whole scene, we traverse the graph recursively, usually in a depth-first manner.

JavaScript Pseudo-code

```
// A recursive function to draw the scene graph
function drawNode(node, parentWorldMatrix) {
  // 1. Calculate this node's world matrix
  const nodeWorldMatrix = mat4.create();
  mat4.multiply(nodeWorldMatrix, parentWorldMatrix, node.localMatrix);

  // 2. If this node has geometry, draw it
  if (node.geometry) {
    gl.uniformMatrix4fv(modelMatrixLoc, false, nodeWorldMatrix);
    drawObject(node.geometry);
  }

  // 3. Recursively call for all children
  for (const child of node.children) {
    drawNode(child, nodeWorldMatrix);
    // Pass our matrix down
  }
}

// Initial call in main loop:
drawNode(scene.root, identityMatrix);
```

Code: The Unchanged Vertex Shader

Crucially, our vertex shader remains simple. It knows nothing about the scene graph.

GLSL Vertex Shader

```
// Vertex Shader
uniform mat4 u_projectionMatrix;
uniform mat4 u_viewMatrix;
uniform mat4 u_modelMatrix; // The final calculated world matrix
attribute vec4 a_position;

void main() {
    // The GPU's job is always the same: transform one vertex
    // using the final matrices it receives.
    gl_Position = u_projectionMatrix * u_viewMatrix * u_modelMatrix *
        a_position;
}
```

Analysis

The GPU receives the final, pre-calculated Model Matrix for each object. This allows it to do what it does best: process thousands of vertices in parallel without worrying about the scene's logical structure.

While understanding the matrix math is vital, high-level libraries like **Three.js** provide an intuitive API that handles the scene graph complexity for us.

Mapping Concepts to Three.js API

- **A Node** → `new THREE.Object3D()` or `Mesh`
- **Parent-Child Relationship** → `parent.add(child)`
- **Local Transformation** → Modifying `.position`, `.rotation`, `.scale`
- **Graph Traversal** → Handled automatically by `renderer.render(scene, camera)`

This lets us focus on the *what* (building a creative scene) instead of the *how* (manual matrix math and recursion).

Three.js Example: A Solar System

JavaScript Code

```
// 1. CREATE NODES: Each mesh is a node
const sun = new THREE.Mesh(sphereGeom, sunMaterial);
const earth = new THREE.Mesh(sphereGeom, earthMaterial);
const moon = new THREE.Mesh(sphereGeom, moonMaterial);

// 2. BUILD THE HIERARCHY (The Scene Graph)
scene.add(sun);
// Sun is a child of the root scene
sun.add(earth); // Earth is a child of the Sun
earth.add(moon); // Moon is a child of the Earth

// 3. SET LOCAL TRANSFORMATIONS
earth.position.x = 10; // Position Earth relative to the Sun
moon.position.x = 2; // Position Moon relative to the Earth

// 4. ANIMATION LOOP
function animate() {
    sun.rotation.y += 0.005; // Sun spins on its axis
    earth.rotation.y += 0.01; // Earth spins on its axis

    // The hierarchy makes Earth orbit the Sun, and Moon orbit Earth!
    renderer.render(scene, camera);
}
```

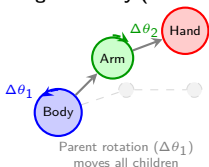
Applications of the Scene Graph

Beyond Simple Placement: Construction, Motion, and Speed

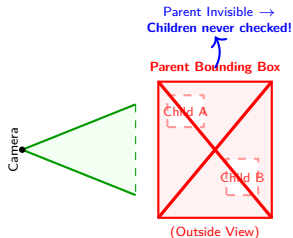
The Scene Graph is the backbone of modern graphics engines, serving multiple critical roles beyond just storing objects.

Application	Concept & Role	Key Benefit / Formulation
1. Model Construction	Composition: Building complex objects from simpler parts (e.g., a Car = Chassis + 4 Wheels + Lights).	Relative Positioning: $M_{world} = M_{parent} \cdot M_{local}$
2. Animation (Forward Kinematics)	Propagation: Motion applied to a parent (e.g., Shoulder) automatically flows to children (Elbow, Hand).	Chain Rule: $M_{hand} = M_{shoulder} \cdot M_{elbow}$
3. Optimisation (Frustum Culling)	Spatial Pruning: Bounding Volume Hierarchies (BVH). If a parent node is invisible, all its children are skipped.	Performance: $O(\log n)$ checks vs $O(n)$.
4. Scene Management	Logical Grouping: Organising entities for batch operations (e.g., "All Enemies", "Transparent Objects").	Recursive Traversal: <code>node.traverse(fn)</code>

Visualising Hierarchy (Kinematics)



Visualising Optimisation (Culling)



What is a Scene Graph?

It is a hierarchical data structure (a DAG^a) that organizes all objects in a 3D scene, using parent-child relationships to manage complexity.

^aDirected Acyclic Graph: A graph structure where edges have a direction (parent $\rightarrow$ child) and no loops (cycles) exist.

How do Transformations Work?

An object's final **World Transformation** is calculated by multiplying its **Local Transformation** with its parent's world transformation. This process is called concatenation.

The Big Picture

The scene graph is a **CPU-side abstraction** to make our lives easier. It calculates the final Model Matrix for each object, which is then sent to the GPU for rendering. The GPU itself is unaware of the hierarchy.

Thinking in Hierarchies

From Human Intuition to Silicon Reality

1. The Philosophy

Stop calculating absolute coordinates (x, y, z). Start defining **relationships** (Parent $\rightarrow$ Child). *"The Arm doesn't know where the World is; it only knows where the Body is."*

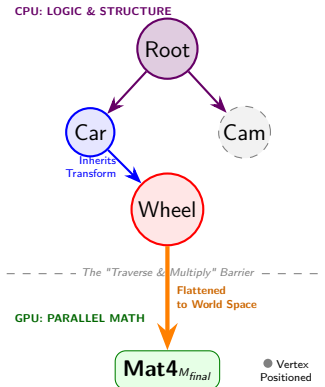
2. The Golden Equation

$$M_{world} = M_{parent} \times M_{local}$$

- **Compose:** Multiply matrices up the tree.
- **Reuse:** Move the Parent, and the Children follow for free.

3. The Architecture

The **CPU** manages the logic (The Graph). The **GPU** receives the result (The Matrix).



"We build the world as a Tree, but we draw it as a List."

Thank you! Any questions?

Study Guide: Mastering Scene Graphs

A Roadmap from Theory to Implementation

1. Mental Model (The "Why")

Focus on the **Proprioception Paradox** . Don't think in global coordinates; think in **Relationships**. If a car moves, why should the wheels manually follow?

2. Mathematical Foundation

Master the **Matrix Concatenation** formula:

$$M_{world} = M_{parent_world} \times M_{local}$$

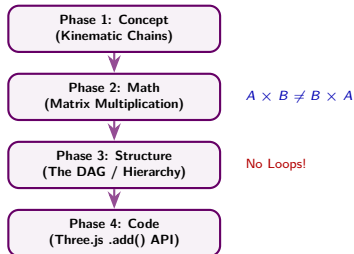
Understand that transformations propagate from **Root to Leaf** .

3. The System Architecture

Distinguish between **CPU** (Hierarchy/Logic) and **GPU** (Flat Math/Shaders) . The GPU never sees the "tree".

4. Practical Implementation

Compare **Manual WebGL** (you are the engine) vs. **Three.js** (you are the architect).



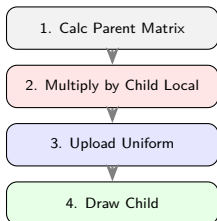
Appendix: WebGL vs. Three.js

Procedural Ordering vs. Object Hierarchy

Raw WebGL: "Implicit Hierarchy"

WebGL does not have a scene graph. It only knows matrices.

- **Mechanism:** Statement Reserved Ordering.
- **How it works:** You must manually multiply matrices in the correct order in your JavaScript code before every draw call.
- **Risk:** If you get the multiplication order wrong, the hierarchy breaks.

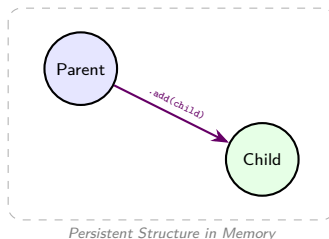


Manual Flow
(Repeats per object)

Three.js: "Explicit Hierarchy"

Three.js abstracts the math into objects.

- **Mechanism:** `.add()` / `.children` array.
- **How it works:** You define the **structure** once. The engine handles the multiplication order automatically during the render loop.
- **Benefit:** Guaranteed consistency.



Appendix: Implementation Code Comparison

Comparing the logic flow for Scene Graph Construction (WebGL vs. Three.js)

Raw WebGL (Manual Math)

```
// 1. Define Parent Transform (Global)
const parentWorld = mat4.create();
mat4.translate(parentWorld, parentWorld, [10,0,0]);

// 2. Define Child Transform (Local)
const childLocal = mat4.create();
mat4.rotateY(childLocal, childLocal, 0.5);

// 3. MANUAL HIERARCHY CONSTRUCTION
const childWorld = mat4.create();

// ORDER MATTERS: Parent * Child
// This reads right-to-left:
// "Take childLocal, THEN apply parentWorld"
mat4.multiply(childWorld, parentWorld, childLocal);

// 4. Send the *Result* to the GPU
gl.uniformMatrix4fv(u_Model, false, childWorld);
gl.drawArrays(...);
```

↑ The hierarchy exists *only* in that multiply line.

Three.js (Graph API)

```
// 1. Create Objects (Nodes)
const parent = new THREE.Mesh(geom, mat);
const child = new THREE.Mesh(geom, mat);

// 2. THE HIERARCHY IS DEFINED HERE:
// Declarative structure
parent.add(child);

// 3. Set Local Transforms
parent.position.set(10, 0, 0);

// 4. Render
// Engine automatically calculates matrices
renderer.render(scene, camera);
```

↑ The structure persists across frames.

Key Difference: WebGL requires you to act as the *Traversal Engine*. Three.js lets you act as the *Architect*.

Appendix: Selective Rendering

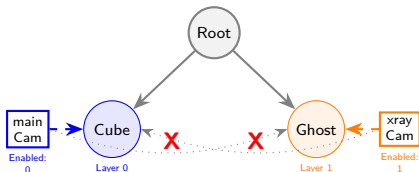
Using Layers to Filter the Scene Graph

Sometimes we need to render **only a subset** of the Scene Graph (e.g., hiding UI objects from a reflection camera, or showing "X-Ray" objects to a specific camera).

The Logic: Channels

Every object and Camera in Three.js has a `.layers` property (a bitmask 0-31).

- An object is drawn **only if** it shares a layer with the active Camera.
- This allows a single Scene Graph to serve multiple views (Minimap, Main View, Bloom Pass) simultaneously.



Three.js Implementation (.layers)

```

// 1. Setup Objects
const cube = new THREE.Mesh(geo, mat);
const ghost = new THREE.Mesh(geo, mat);

// Assign channels (0 is default)
cube.layers.set(0);
ghost.layers.set(1); // "Hidden" layer

// 2. Setup Cameras
const mainCam = new THREE.PerspectiveCamera();
mainCam.layers.enable(0); // Sees Cube
mainCam.layers.disable(1); // Ignores Ghost

const xrayCam = new THREE.PerspectiveCamera();
xrayCam.layers.enable(1); // Sees Ghost

// 3. Render Loop
function render() {
    // Pass 1: Draw solid world
    renderer.render(scene, mainCam);

    // Pass 2: Draw ghosts (e.g. for glow)
    renderer.autoClear = false;
    renderer.render(scene, xrayCam);
}

```

Why use this?

It filters the graph **during traversal**. The renderer skips non-matching nodes entirely, saving GPU cost without removing them from memory.

Appendix: Filtering by Object Type

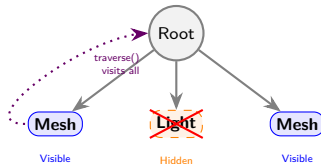
Dynamic Traversal for Adaptive Views

The Scene Graph structure allows us to **traverse** (visit) every node. We can use this to adaptively filter the scene, such as hiding all lights or showing only specific helpers.

The Logic: Traversal

Three.js objects have internal boolean flags (e.g., `.isMesh`, `.isLight`).

- We can write a recursive function to visit every descendant.
- If a node matches our criteria, we toggle its `.visible` property.



Three.js Implementation (`.traverse`)

```

// Function to toggle visibility by Type
function filterScene( scene, typeKey, isVisible ) {

    // .traverse() runs the callback on the node
    // AND all its children recursively.
    scene.traverse( function( node ) {

        // Check internal Three.js flags
        // (e.g., node.isMesh, node.isLight)
        if ( node[ typeKey ] === true ) {
            node.visible = isVisible;
        }

    });

}

// USAGE EXAMPLES:

// 1. Hide all Lights (e.g. for "Unlit" mode)
filterScene( scene, 'isLight', false );

// 2. Show all Meshes
filterScene( scene, 'isMesh', true );

// 3. Hide Debug Helpers
filterScene( scene, 'isAxesHelper', false );

```

Why use this?

Unlike Layers (which filter continuously), this is a **one-time state change**. Useful for switching game modes (e.g., "Editor Mode" vs "Play Mode").